Full System Plan — Global Intelligence Ecosystem

Master plan v1 · 2026-10-25 · built entirely from scratch · final system name pending

One data layer → three parallel engines → two private power stacks → two public money machines → a content engine that multiplies everything into reach. Collect once, organize properly, reuse many times, control access separately.

The Three Rows — complete architecture

ROW 1 — PUBLIC PRODUCTS & MEDIA

revenue + reach

🖥️ Terminal

Bloomberg-class public intelligence platform: scanners, fundamentals, technicals, quality scores, charts, research, subscriptions, public APIs. Consumes ALL 3 engines directly.

📰 News portal

Worldwide news — every niche & domain (not just finance): summaries, timelines, source refs, credibility scores. Consumes ALL 3 engines directly.

🎬 Media hub

LAST in Row 1 YouTube / Instagram / social / sites / entertainment. Consumes ONLY Terminal + Portal outputs — never touches the engines (rights firewall).

▲ approved views only — the Terminal/Portal boundary is the ONLY public gateway to engine data

ROW 2 — THREE HEADLESS INTELLIGENCE ENGINES

built FIRST · feeds everything

💹 Financial Engine

collection → historical archives → realtime ingestion → quality scoring → time-series storage → replay → APIs + event streams

🗞️ News Engine

continuous collection → timelines → verification + credibility scoring → cross-source linking

🌍 World Engine

geo, registries, shipping, satellites, events, macro — world-monitoring collection

⚙️ shared machinery

source registry (1,000-dataset catalogue seed) · schemas + provenance · quality scoring · storage lanes · event bus · APIs · access control + data-rights · monitoring

🧠 AI-assisted organization

entity resolution · gap detection · anomaly detection · source discovery → turns collected data into CONNECTED intelligence

ROW 3 — PRIVATE OPERATING LAYER

all 3 engines · two separate stacks

💼 FinOS — trading & management stack

all 3 engines → quant research → backtest (PIT replay) → risk gateway → controlled live execution → portfolio & risk. Non-LLM RL agents live here (train→eval→paper→approval→deploy). HFT = separately engineered low-latency path.

🛰️ Cockpit layer

planning, agents, missions, memory, approvals, governance, software & business automation. Coordinates everything — never sits in fast data/trading paths.

🕵️ World Monitor + personal AI

LAST IN ENTIRE PROJECT Fuses ALL 3 engines into one private surveillance & investigation surface per entity/event/time — dashboard + investigation workflows + personal intelligence on top.

Why it is an ecosystem (not a product list)

One news event collected once by an engine → **FinOS signal** trading input → **Portal story** public revenue → **YouTube explainer** brand + reach → **World Monitor case** investigation

The engines are **the asset**. Products, channels, desks and dashboards are faces over the same collected intelligence. Every layer feeds another.

Non-negotiable architecture rules

R1 · Collect once, reuse many — consumers subscribe to validated engine output; no hub ingests raw upstream directly.

R2 · Rights firewall — Row 1 products are the ONLY gateway to the public; license classes (free / internal-only / no-local-store / never-commercial) enforced at the boundary.

R3 · Intelligence ≠ fast paths — the cockpit plans and governs; time-sensitive market flow runs deterministic private lanes.

R4 · PIT & survivorship — backtest data must be as-of-filing/vintage; overwrite-in-place vendors mechanically fenced.

R5 · Provenance everywhere — every record carries source-URL, fetch time, parser version, rights class (Two-Clicks lineage at schema level).

R6 · Versioned event contracts — consumers depend on contracts, never on implementations.

R7 · RL agent gates — train → evaluate → paper → approval → deploy; agents never touch their own risk limits.

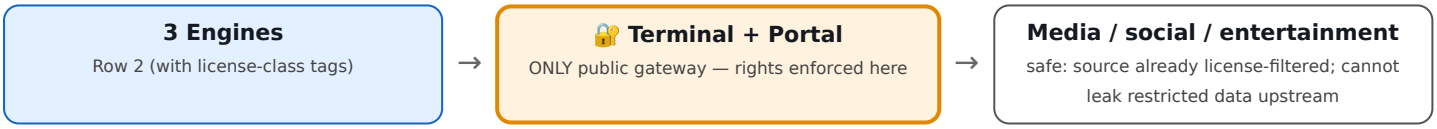
R8 · HFT honesty — dedicated subsystem, benchmarked separately (P50/95/99, gaps, recovery); retail feeds are research-grade.

The four communication contracts

Method	Use	Boundary controls
Request/response API	point queries (latest price, news t1..t2)	auth · quotas · rights
Streaming subscription	continuous ("send every relevant update")	per-conn budget · latency class
Event notification	"new filing arrived" · "source died"	queue perms · retention
Bulk transfer	10-year research loads	throughput caps · never blocks live paths

Latency targets (benchmark, not claims): Rust in-process µs–sub-ms · optimized local service sub-ms · local event bus low-ms · indexed historical query ms–100s · AI/verification async (seconds+) · bulk = throughput-first. Always separate source delay from our delay.

Rights firewall — how public data escapes (and how it can't)



Direct engine → media connections: forbidden by design. Engine outage ≠ content outage (public products cache and buffer).

Foundation design decisions (finalize at Stage-1 kickoff)

Workspace — monorepo: Rust core (ingestion, event bus, hot paths) + Python edge (quant research, analytics); hard language boundary.

Storage lanes — parquet lake (archives) · time-series engine for tick (evaluate QuestDB/Timescale/ClickHouse) · event log (NATS/Redis Streams first) · reference store · different DBs for different workloads.

Registry seed — repot the 1,000-dataset research catalogue (license / verification / latency / history depth) into the source-registry schema.

Naming — all responsibility-names above are placeholders; renamed uniformly when the owner announces the final system name.

Build order — 10 stages, dependency-true

Vertical slices (one dataset family fully through the stack) beat horizontal layer-building. Two hard dependencies: World Monitor (8) after FinOS (5) · Media (9) after Terminal+Portal (7).

1. **Foundation skeleton**
   source registry · schema+provenance contracts · storage lanes · event bus · APIs · auth/quotas/data-rights · monitoring
   — feeds everything
2. **Financial Engine v0**
   collection · archives · realtime · quality scoring · replay · query APIs (benchmark bundle → India EOD/F&O → crypto tails)
   — FinOS early · Terminal later
3. **News Engine v0 + World Engine v0**
   document/event payloads · timelines · evidence + credibility scoring · geo/event collection
   — Row 1 · World Monitor · FinOS signals
4. **AI-assisted organization**
   entity resolution · gap detection · anomaly detection · source discovery (moat classes pay off here)
   — prereq for World Monitor
5. **FinOS v0→v1**
   research env → backtester on PIT replay → paper → risk gateway → controlled execution
   — private revenue + capability
6. **Cockpit layer**
   missions · agents · approvals · memory · governance over both stacks
   — operations scale
7. **Terminal + News portal launch**
   public products on approved views · subscriptions · public APIs · editorial workflows
   — public revenue · feeds Media
8. **World Monitor** fuses all 3 engines + personal AI — surveillance & investigation surface
   — **LAST IN ENTIRE PROJECT**
9. **Media/entertainment hub** multi-platform content from Terminal+Portal outputs only
   — **LAST in Row 1**
10. **Scale & specialize** throughput · historical coverage · cross-engine intelligence · RL expansion · HFT vs measured requirements
    — matures everything

Honest risks & standing constraints

Risk	Handling
Blocked-fetch frontier (anti-bot walls, 403/429, JS-only portals)	rows carry real verify/quote actions; system never silently skips
Entity resolution is research-grade hard	its own stage (4) — not a weekend side quest
Single-box fragility	backup story for raw archives BEFORE 100s-of-GB backfills land
Licensing drift on paid/walled rows	re-verification cadence; rights enforced at boundaries
Latency claims	engineering targets only — benchmark P50/95/99, gaps, recovery
Trading outcomes	fast ≠ profitable — strategy quality, liquidity, costs decide

Final line: you are building the infrastructure, the storefronts, and the trading desk from ONE shared foundation — so every product, strategy, and story reuses the same collected intelligence.